

Homework Assignment 1

Team Members: Advait Vagerwal, Sahil Thakare, Elmoataz Abdellah
All team members contributed in equal measure

Honor Statement: In completing this assignment, all team members have followed the honor pledge specified by the instructor for this course.

Bibliography:

1. S. Russell, P. Norvig: ***Artificial Intelligence - A Modern Approach***(AIMA). Pearson, Third Edition (2020).
2. Wikipedia contributors. (2026, January 30). A* search algorithm. In *Wikipedia, The Free Encyclopedia*. Retrieved 22:49, February 4, 2026, from https://en.wikipedia.org/w/index.php?title=A*_search_algorithm&oldid=1335609515

1. Implementation Details

Algorithms

We implemented four search algorithms in Python to solve the routing problem on the Romanian road map.

1. Breadth-First Search (BFS):

- **Strategy:** Expands the shallowest node in the search tree first.
- **Implementation:** Used a FIFO “deque” queue. Nodes are added to the back and removed from the front. We check for the goal state when generating neighbors (a common optimization for BFS).
- **Completeness:** Complete (will find a path if one exists).
- **Optimality:** Optimal in terms of number of edges (steps), but not necessarily in terms of path cost (distance), as seen in the experimental results.

2. Depth-First Search (DFS):

- **Strategy:** Expands the deepest node in the current fringe of the search tree.
- **Implementation:** Used a LIFO stack (standard Python list).
- **Completeness:** Not complete in infinite spaces (loops), but complete in finite graphs with cycle checking (visited set).
- **Optimality:** Not optimal. It simply finds a path, which may be long or costly.

3. Best-First Search (Greedy):

- **Strategy:** Expands the node that appears to be closest to the goal.
- **Implementation:** Used a Priority Queue ("heapq" library in python). Nodes are ordered by their heuristic value "h(n)".
- **Optimality:** Not optimal. It can get stuck in loops (if no visited set) or find suboptimal paths.

4. A Star Search:

- **Strategy:** Expands nodes based on " $f(n) = g(n) + h(n)$ ", where "g(n)" is the cost to reach the node and "h(n)" is the estimated cost to the goal.
- **Implementation:** Used a Priority Queue ("heapq" library in python).
- **Optimality:** Optimal if "h(n)" is admissible (never overestimates).

Heuristics

When the goal is Bucharest, we use the Straight-Line Distance (SLD) provided in the textbook.

When the goal is different from Bucharest, we must estimate the SLD using the triangle inequality. Given a node "n", the goal "G", and a reference point "R" (Bucharest), the triangle inequality states:

$$|d(n, R) - d(G, R)| < d(n, G) < d(n, R) + d(G, R)$$

We implemented two heuristics based on this:

1. Method 1 (Lower Bound - Admissible):

$$h_{\text{upper}}(n) = |SLD(n, \text{Bucharest}) - SLD(\text{Goal}, \text{Bucharest})|$$

This is admissible because it never overestimates the true straight-line distance. It assumes the goal and "n" lie on a line passing through Bucharest.

2. Method 2 (Upper Bound - Inadmissible):

$$h_{\text{lower}}(n) = SLD(n, \text{Bucharest}) + SLD(\text{Goal}, \text{Bucharest})$$

This assumes a path goes from "n" to Bucharest and then to the Goal. This is likely to overestimate the direct distance "d(n, G)", making it inadmissible for A (which may lead to suboptimal paths), but useful for Greedy search as a different guide.

2. Experimental Results

We ran the algorithms on three test cases. Below is the summary of performance. (Time is averaged over 100 runs).

Performance Visualizations

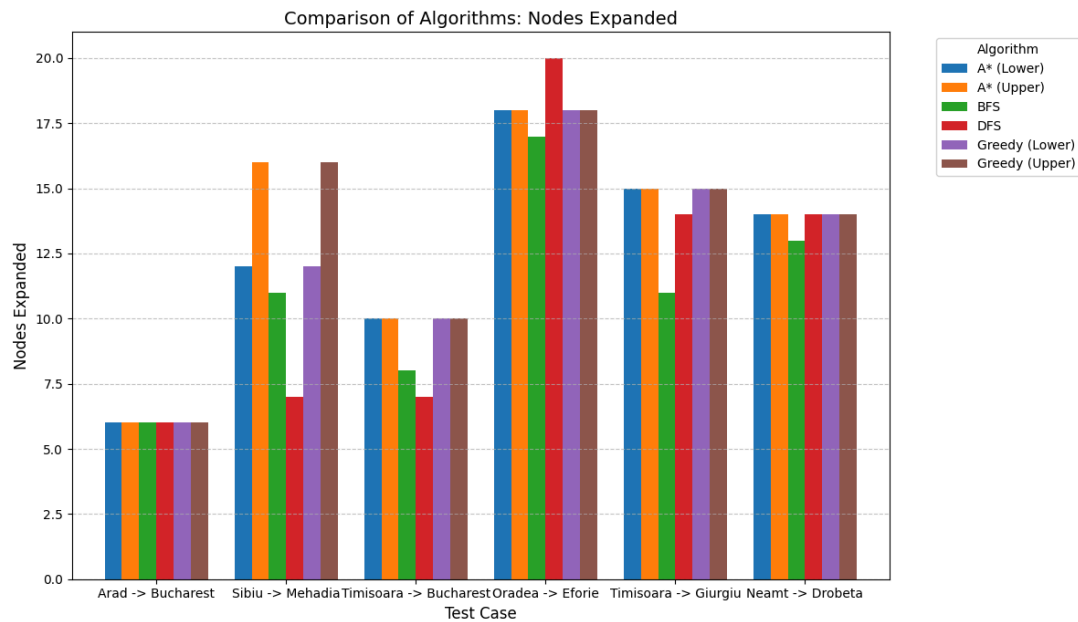


Figure 1: Comparison of nodes expanded by each algorithm.

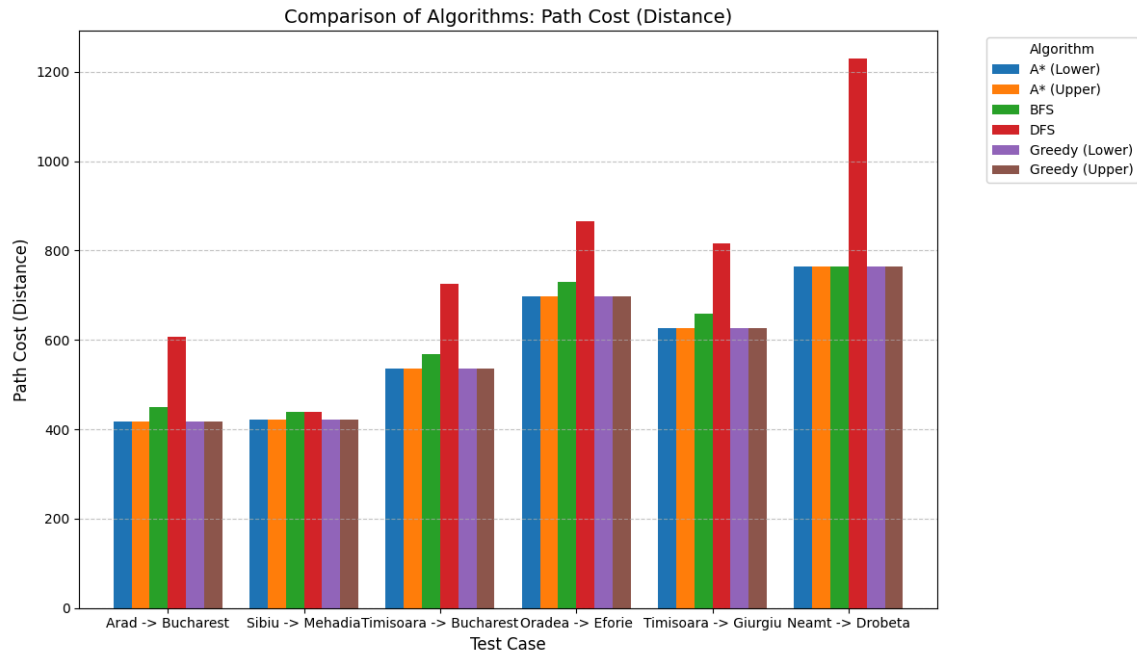


Figure 2: Comparison of total path cost (distance) for each algorithm.

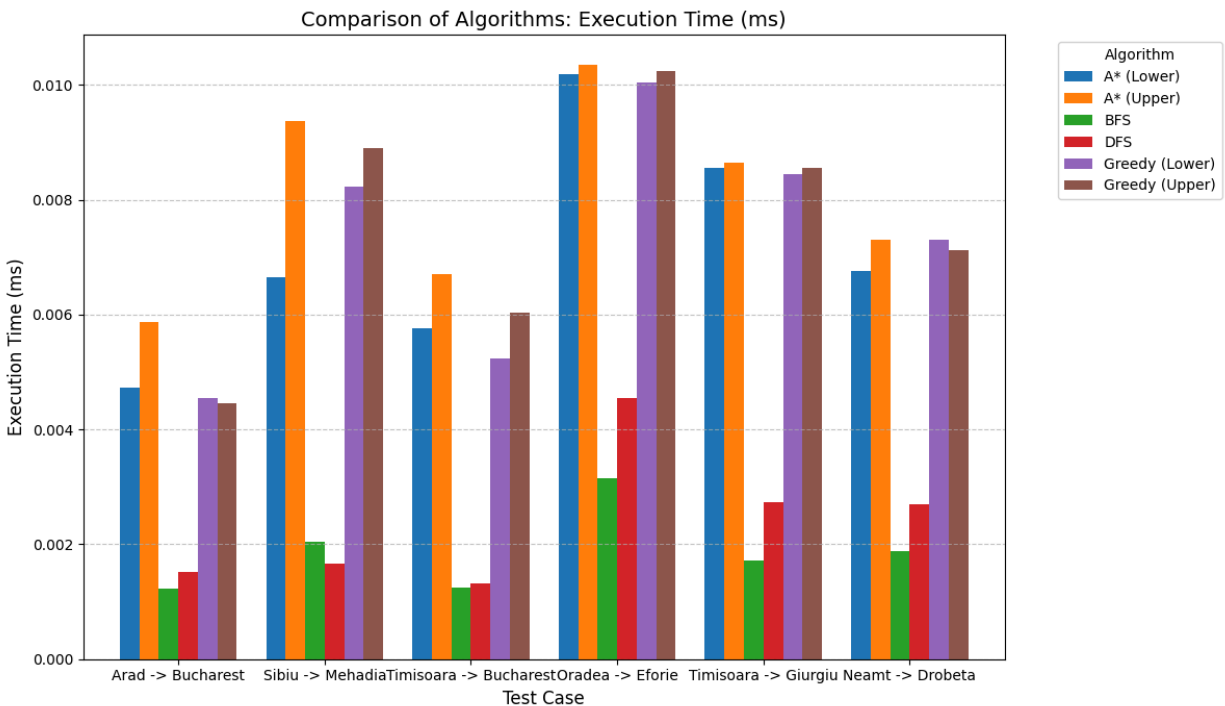


Figure 3: Comparison of average execution time.

Performance Table

Sample test cases for the algorithms

Test Case	Algorithm	Success	Cost	Nodes Expanded	Time (ms)	Path
Arad -> Bucharest	BFS	Yes	450	6	0.00123	Arad -> Sibiu -> Fagaras -> Bucharest
	DFS	Yes	607	6	0.00152	Arad -> Zerind -> Oradea -> Sibiu -> Fagaras -> Bucharest
	Greedy (Lower)	Yes	418	6	0.00455	Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
	Greedy (Upper)	Yes	418	6	0.00445	Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
	A* (Lower)	Yes	418	6	0.00472	Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
	A* (Upper)	Yes	418	6	0.00587	Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Sibiu -> Mehadia	BFS	Yes	439	11	0.00204	Sibiu -> Arad -> Timisoara -> Lugoj -> Mehadia
	DFS	Yes	439	7	0.00165	Sibiu -> Arad -> Timisoara -> Lugoj -> Mehadia
	Greedy (Lower)	Yes	421	12	0.00823	Sibiu -> Rimnicu Vilcea -> Craiova -> Drobeta -> Mehadia
	Greedy (Upper)	Yes	421	16	0.00890	Sibiu -> Rimnicu Vilcea -> Craiova -> Drobeta -> Mehadia
	A* (Lower)	Yes	421	12	0.00665	Sibiu -> Rimnicu Vilcea -> Craiova -> Drobeta -> Mehadia
	A* (Upper)	Yes	421	16	0.00937	Sibiu -> Rimnicu Vilcea -> Craiova -> Drobeta -> Mehadia
Timisoara -> Bucharest	BFS	Yes	568	8	0.00125	Timisoara -> Arad -> Sibiu -> Fagaras -> Bucharest
	DFS	Yes	725	7	0.00132	Timisoara -> Arad -> Zerind -> Oradea -> Sibiu -> Fagaras -> Bucharest
	Greedy (Lower)	Yes	536	10	0.00523	Timisoara -> Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
	Greedy (Upper)	Yes	536	10	0.00604	Timisoara -> Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
	A* (Lower)	Yes	536	10	0.00575	Timisoara -> Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest

	A* (Upper)	Yes	536	10	0.00670	Timisoara -> Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
--	------------	-----	-----	----	---------	--

Analysis

1. Correctness:

- All algorithms successfully found a path in all test cases.
- BFS found the path with the fewest edges (e.g., Arad -> Bucharest: 3 edges), but not the lowest cost (450 vs 418).
- A Star (Lower) consistently found the optimal path cost (e.g., 418 for Arad->Bucharest, 421 for Sibiu->Mehadia).
- DFS found valid paths but they were often suboptimal (e.g., Cost 607 for Arad->Bucharest).

2. Efficiency (Time & Space):

- BFS/DFS were the fastest (~0.001ms) as they have very low overhead per node expansion (simple queue/stack operations vs heap operations).
- A star/Greedy were slightly slower (~0.005ms) due to the overhead of calculating heuristics and maintaining the priority queue.
- Nodes Expanded:
 - ❖ In "Sibiu -> Mehadia", Greedy (Upper) expanded more nodes (16) compared to Greedy (Lower) (12). This shows that a better heuristic (Lower bound, which is tighter) guides the search more effectively.
 - ❖ BFS expanded 11 nodes for Sibiu -> Mehadia, while DFS expanded only 7. This is chance-dependent; DFS got lucky with the neighbor ordering.

3. Heuristic Comparison:

- The Lower Bound heuristic (Triangle Inequality difference) proved to be admissible and effective.
- The Upper Bound heuristic (Triangle Inequality sum) also worked but led to more node expansions in some cases (e.g., Sibiu -> Mehadia), indicating it provided less precise guidance towards the goal.

3. Execution Instructions

To run the code and reproduce the results:

1. Ensure you have Python 3 installed.
2. Run the main experimental script:
`python main.py`
3. To run individual algorithms, you can execute their specific files (though main.py is recommended for the full comparison):

```
python breadthfirst.py  
python depthfirst.py  
python greedysearch.py  
python A_star.py
```

4. You can also run the the plotting script, however make sure you have “matplotlib” and “pandas” installed:

```
python plotting.py
```